



Universidad Nacional General Sarmiento

Trabajo Práctico: Super Elizabeth Sis

Integrantes:

- Melanie Romero (47309044) - melanieromero7111@gmail.com
- Santiago Velázquez (46875648) - santiagonahuelvelazquez05@gmail.com
- Candela Gullo (46355554) - candegullo22@gmail.com

Cátedra:

Programación 1 (Comisión - 03)

Profesores:

Lucas Bidart.

Leonardo Dávalos

Link repo: <https://github.com/yoongi-programmer/gullo-romero-velazquez-tp-pl>

Introducción

Para este trabajo práctico desarrollamos un videojuego de plataformas en Java utilizando la biblioteca Entorno. El juego tiene como protagonista a una princesa que debe recorrer un mapa formado por distintas islas flotantes mientras evita caer al vacío y enfrenta enemigos que aparecen a lo largo del recorrido.

A medida que avanza, la princesa puede desplazarse, saltar entre plataformas y disparar proyectiles para eliminar a los enemigos. Algunos de ellos pueden dejar caer poderes especiales que permiten mejorar temporalmente los ataques del personaje o recuperar vidas. Además, luego de derrotar una cierta cantidad de enemigos aparece un jefe final con comportamientos y mecánicas diferentes al resto, lo que agrega un desafío extra al juego.

El objetivo principal es sobrevivir a los obstáculos, superar a los enemigos y llegar hasta el castillo ubicado al final del mapa para ganar la partida. Durante el desarrollo se implementaron distintas funcionalidades como movimiento con física básica, gravedad, colisiones, generación de enemigos, disparos, poderes especiales, desplazamiento del mapa y un sistema de vidas.

Este proyecto permitió aplicar conceptos fundamentales de programación orientada a objetos, utilizando clases, objetos, atributos, métodos y la interacción entre distintos componentes del juego para construir una experiencia interactiva y dinámica.



Descripción de las clases

Clase Juego

La clase **juego** es la clase principal de este proyecto, como su nombre lo denota. Su función principal es controlar y dirigir todos los objetos instanciados para que funcionen conectados, como un director de orquesta. Manejamos un sistema de estados del juego para poder mostrar distintas pantallas como un menú, función de pausa, pantalla de victoria y de derrota. Son cinco estados (Menú, Juego, Pausa, Ganó y Perdió). Todas las cálculos y mecánicas del juego viven dentro del segundo estado, donde se aplica constantemente gravedad al personaje de la princesa y se valida si sus pies tocan alguna isla para desplazarse, se detectan las interacciones del teclado y mouse en el juego para tomar decisiones cómo reiniciar el juego, poner pausa, disparar etc. Y se resta vidas al personaje cuando colisiona con enemigos o cae al vacío, de forma que se pueda finalizar el juego.

Variables de instancia:

- **entorno:** instancia que nos permite usar los métodos del paquete entorno
- **ESTADOS:** se usa como constantes para guardar los estados del juego (Menú, Pausa, Jugando, etc.)
- **islas:** almacena las islas en un arreglo de la clase **Isla**.
- **princesa:** instancia de la clase **Princesa**, personaje principal
- **castillo:** instancia de la clase **Castillo**, sirve como bandera para ganar
- **jefe:** instancia de la clase **Jefe**, representa a un enemigo final
- **enemigos:** instancia de la clase **Enemigo** ad
- **poder:** instancia de la clase **Poder** que sirve como power up para la protagonista
- **tienePoder:** indicador booleano para activar el power up
- **jefeInvocado:** indicador booleano para decidir si dibujar al jefe o no
- **tocandoElPiso:** indicador para dejar de aplicar "gravedad" a la princesa y permitir saltar solo cuando esté sobre una isla
- **disparosEspeciales:** contador de los disparos especiales que se le habilitan cuando tiene un poder

- **ganar:** bandera para terminar el juego cuando gana
- **xMapa** e **yMapa:** posición x e y del mapa del juego, se utiliza para el desplazamiento del mapa.
- **imgPantallas:** variables para guardar la ruta de las imágenes del menú, pausa, ganar, etc.
- **musicaEstados:** indicadores booleanos para saber cuándo reproducir qué música según el estado del juego.

Métodos principales:

- **dibujar(e, img, x, y, esc):** método auxiliar que dibuja una imagen.
- **pantallaMenu():** cambia la pantalla por la de un menú antes del juego
- **pantallaPausa():** muestra en pantalla un fondo de pausa con indicaciones para volver al juego o reiniciar
- **dibujarIndicadores(cant, posX, img, esp,esc):** dibuja en la parte superior de la pantalla dos indicadores gráficos, uno para las vidas y otro para los disparos especiales
- **inicializarPiso():** inicializa las plataformas en forma de islas para que la princesa se desplace.
- **generarIslasFlotantes():** genera las plataformas flotantes dibujadas como islas y con posiciones aleatorias pero jugables.
- **moverMapa():** desplaza el fondo del mapa y todos los personajes del juego que se deban mover con el mapa cuando se mueve la princesa.
- **reiniciarJuego():** resetea todas las variables importantes del juego a su valor inicial para volver a jugar.
- **tick():** representa un instante de tiempo por lo que se encarga de actualizar el estado interno del juego. Maneja los estados del juego, el principal es el de "Jugando", que maneja las colisiones entre los distintos personajes, la generación del mapa, desplazamiento de la princesa, etc.

Problemas y soluciones:

El mayor problema acá fue que la clase juego era una que todos modificamos y tocamos, por más que se distribuyeran las tareas, por lo que para subir los cambios hubieron muchos merge de github y se pisaban algunos cambios. A pesar de todo se pudo integrar bien los cambios.

Respecto del juego, algunos problemas fueron generar las islas flotantes aleatorias pero con cierto control para que sea jugable, para lo que se hizo un método con muchas validaciones a los candidatos de posiciones para las islas antes de dibujarlas. Se añadieron distintas músicas de fondo para cada estado del juego por lo que fue un problema también cómo hacer que se reproduzca una, termine y se reproduzca la siguiente canción.

Clase Princesa

La clase **Princesa** representa a la protagonista del juego, la cual es controlada directamente por el usuario. Su función principal es permitir al jugador interactuar con el entorno a través de movimientos laterales, saltos entre islas flotantes y el lanzamiento de proyectiles en forma de defensa contra los enemigos del juego.

Variables de instancia:

- **x,y:** determinan las coordenadas de posición de la princesa en el mapa.
- **ancho, alto:** dimensionan el tamaño de la princesa y el área de colisión.
- **velocidad:** determina la velocidad del movimiento horizontal.
- **vidas:** funciona como contador que almacena la “salud” o intentos restantes de la princesa
- **mirandoDerecha:** indicador booleano utilizado para definir la orientación visual de la princesa
- **framesIzquierda, framesDerecha:** arreglos de imágenes utilizados para almacenar las secuencias de animación de carrera.
- **disparo:** referencia al proyectil activo lanzado por la princesa.
- **frameActual:** cuadro de la animación que se está mostrando en el momento.
- **contadorDeAnimacion:** controla la velocidad en que se refresca las imágenes de movimiento.
- **tocandoElSuelo:** Indica si la princesa está apoyada sobre el suelo.
- **gravedad, velocidadY, potenciaSalto:** variables encargadas de simular la caída libre y el impulso vertical de forma más realista.

Métodos principales:

- **Princesa()**: el constructor se encarga de posicionar inicialmente al personaje, configurar sus atributos de movimiento, físicas base y realizar la carga indexada de las imágenes para las animaciones en ambas direcciones.
- **paradaSobreIsla(Isla isla)**: evalúa si los pies de la princesa hacen contacto con la superficie superior de una plataforma flotante en pleno descenso, deteniendo su caída.
- **chocaCabezaConIsla**: detecta colisiones verticales ascendentes contra la base inferior de las islas para frenar el impulso del salto.
- **chocaLadoDerechoConIsla(Isla isla) / chocaLadoIzquierdoConIsla(Isla islas)**: comprueban las colisiones laterales con las paredes de las islas para evitar que el personaje las atravesase de forma horizontalmente.
- **moverseDerecha() / moverseIzquierda()**: actualizan la posición horizontal, modifican el sentido de orientación del personaje y avanzan secuencialmente el contador de animación correspondiente.
- **saltar()**: altera la velocidad vertical utilizando la potencia de salto, permite el despegue únicamente si está sobre una isla.
- **modificarFisica()**: actualiza de forma constante la posición vertical y acumula el efecto de la gravedad sobre la velocidad vertical mientras la princesa está saltando.
- **disparar(double mouseX, double mouseY, boolean especial)**: instancia un proyectil de la clase **Disparo** apuntando de forma precisa hacia las coordenadas del mouse, validando que no exista un disparo previo en ese momento.
- **actualizarDisparo(Isla[] islas)**: actualiza de manera iterativa el desplazamiento del proyectil activo y determina su eliminación (se vuelve null) si sale de los márgenes de la pantalla o impacta contra alguna plataforma del escenario.
- **dibujarse(entorno e)**: permite dibujar el cuadro de animación correcto según la dirección de la princesa.
- **Getters y Setters**: se incorporaron los getters y setters necesarios para obtener y modificar los valores de las variables de instancia de la princesa de forma correcta.

Problemas y soluciones:

El mayor desafío con esta clase se dió al momento de desarrollar el sistema de físicas, tanto las colisiones como el salto de forma “realista”, dado que inicialmente la princesa sufría errores de posicionamiento, atravesaba las islas por los laterales o se producían errores al momento de colisionar con el inferior de las islas. Para resolver este apartado, segmentamos la comprobación de las físicas en cuatro métodos independientes (**paradaSobreIsla**, **chocaCabezaConIsla**, **chocaLadoDerechoConIsla** y **chocaLadoIzquierdoConIsla**). Con respecto al salto, se necesitaba una forma que modifique el salto en pixeles pequeños en cada tick para que diera la ilusión de una subida y caída suave al momento de saltar, para eso utilizamos las variables **velocidadY**, **gravedad** y **potenciaSalto** que nos permitieron modificar de forma dinámica la velocidad vertical para que se de una animación fluida. Y por último otro inconveniente fue el desajuste entre las dimensiones que teníamos del cuadro de colisión real y el espacio ocupado por los sprites (las imagenes que les pusimos a los personajes y objetos), de las animaciones, esto se solucionó mediante la inclusión de una compensación fija del eje Y (**ajusteY = 18**) aplicada únicamente al momento de ejecutar el método que dibuja a la princesa.

Clase Isla

La clase **Isla** representa a las plataformas flotantes y el suelo por los que la princesa puede desplazarse y evitar caer por gravedad. Su función principal es la mencionada antes, añade dinamismo y aleatoriedad al juego de forma que ninguna partida es igual a la anterior.

Variables de instancia:

- **imagen**: almacena la ruta de la imagen para dibujar la isla
- **area**: obtengo del rectángulo sus coordenadas y tamaños para las colisiones con los demás objetos del juego

Métodos principales:

- **Isla ()**: el constructor se encarga de que la misma imagen se adapte al tamaño aleatorio de las islas, escalando la imagen.

- **moverIslas()**: desplaza las islas en el eje x cuando el mapa se mueve a la izquierda o derecha
- **dibujar()**: tiene por función dibujar las islas en el mapa.
- **getX, getY**: devuelve la posición x e y. Son importantes para realizar las colisiones con las demás objetos
- **getAncho, getAlto**: devuelven el ancho y el alto del objeto isla.
- **getArea**: devuelve el rectangle para que otras clases utilicen sus atributos.

Problemas y soluciones:

El desafío más grande con esta clase fue primero la generación de las islas en el mapa de forma aleatoria, ya que era necesario controlar bien la posición aleatoria para que no se generen muy cerca una de la otra, fuera del alcance del salto máximo de la princesa o demasiado abajo atrapándola. Otro problema relacionado con esto fue dibujar las islas con imagen en vez de un rectángulo, fue necesario escalar la imagen antes de dibujar cada isla para que encaje perfecto con el tamaño cambiante.

Clase Castillo

La clase **castillo** es utilizada como “bandera” para finalizar el juego con la victoria de la princesa. Su función principal es finalizar el juego con un “Ganaste!” cuando la princesa colisiona con el mismo.

Variables de instancia:

- **x e y**: almacena la ruta de la imagen para dibujar la isla
- **ancho y alto**: obtengo del rectángulo sus coordenadas y tamaños para las colisiones con los demás objetos del juego
- **imagen**: lo utilizo para dibujar el castillo

Métodos principales:

- **dibujar**: muestra el castillo en pantalla
- **mover**: su función es hacer que el castillo se mueva junto con el mapa

- **colisionaConPrincesa**: detecta la colisión con la princesa y enciende la bandera de ganar.

Problemas y soluciones:

Con esta clase particularmente no tuvimos problemas, pues es simple y su función es clara y fácil de implementar.

Clase Disparo

La clase **Disparo** representa los proyectiles lanzados por la princesa. Su función principal es almacenar la posición del disparo, su dirección, velocidad y características especiales.

Variables de instancia:

- **posicionX, posicionY**: almacenan la posición actual del proyectil.
- **velocidadX, velocidadY**: determinan el movimiento del disparo.
- **diametro**: tamaño utilizado para dibujarlo y detectar colisiones.
- **angulo**: almacena la dirección del disparo calculada entre el origen y el punto al que apunta el jugador. Se utiliza para definir su trayectoria y orientar la imagen correctamente.
- **especial**: indica si el disparo fue generado utilizando el poder especial.

Métodos principales:

- **mover()**: actualiza la posición del proyectil según su velocidad.
- **dibujar()**: muestra el disparo en pantalla.
- **colisionaConIsa()**: verifica si impactó contra alguna plataforma.
- **esEspecial()**: informa si el disparo posee propiedades especiales.
- Getters de posición y tamaño utilizados por otras clases para detectar colisiones.

Problemas y soluciones:

Uno de los desafíos fue calcular correctamente la dirección del disparo hacia la posición del mouse. Para resolverlo se utilizó un vector de dirección normalizado,

permitiendo que todos los disparos viajen a velocidad constante independientemente de la distancia al objetivo.

Clase Enemigos

La clase **Enemigos** representa a los enemigos comunes que aparecen durante la partida. Estos se desplazan por las plataformas y pueden ser eliminados por los disparos de la princesa.

Variables de instancia:

- x, y: posición del enemigo.
- velocidad: velocidad de movimiento.
- vaDerecha: indica la dirección actual.
- ancho, alto: dimensiones utilizadas para colisiones.
- frames: imágenes utilizadas para la animación.
- enemigosMuertos: contador global de enemigos eliminados.
- Variables asociadas a la animación de explosión.

Métodos principales:

- **mover()**: controla el desplazamiento del enemigo.
- **dibujar()**: dibuja el enemigo en pantalla.
- **colisionDisparoEnemigo()**: detecta impactos de disparos.
- **colisionaConPrincesa()**: detecta contacto con la protagonista.
- **explotar()**: inicia la animación de explosión.
- **explosionTerminada()**: informa cuándo debe eliminarse.
- **generarEnemigo()**: crea nuevos enemigos en posiciones válidas.
- **generarPoder()**: genera aleatoriamente un poder al matar a 3 enemigos.

Problemas y soluciones:

Uno de los problemas fue evitar que los enemigos aparecieran encima de otros objetos o fuera de las plataformas. Para solucionarlo se implementó un sistema de generación que valida la posición antes de crear cada enemigo.

Clase Poder

La clase **Poder** representa los objetos especiales que pueden aparecer cuando un enemigo es derrotado. Estos objetos pueden otorgar un **poder especial para los disparos** o una **vida extra**, dependiendo del tipo generado aleatoriamente.

Variables de instancia:

- x, y: posición del objeto en el mapa.
- tipo: indica si el objeto es un poder (0) o una vida extra (1).
- ancho, alto: tamaño utilizado para las colisiones.
- frames: almacena las imágenes de la animación.
- frameActual: cuadro de animación que se está mostrando.
- contadorAnimacion: controla la velocidad de la animación.

Métodos principales:

- **Poder(double x, double y):** crea el objeto, define su posición y genera aleatoriamente si será un poder o una vida extra.
- **dibujar(Entorno entorno):** anima y dibuja el objeto en pantalla.
- **colisionaConPrincesa(Princesa princesa):** verifica si la princesa recogió el objeto.
- **moverConMapa(double direccion):** mueve el objeto junto con el desplazamiento del mapa.
- **getTipo():** devuelve el tipo de recompensa que contiene el objeto.
- **Getters y Setters:** permiten acceder y modificar su posición.

Problemas y soluciones:

Inicialmente el juego solo contaba con un tipo de recompensa. Para agregar una vida extra sin crear una nueva clase, se incorporó la variable **tipo**, permitiendo que la misma clase representen distintos objetos. Además, se aumentó la escala de las imágenes de vida para que resultaran más visibles dentro del juego.

Clase DisparoJefe

La clase **DisparoJefe** representa los proyectiles lanzados por el jefe final durante la fase de combate.

Variables de instancia:

- x, y: posición del disparo.
- velocidad: velocidad de caída.
- frames: imágenes de animación.
- frameActual y contadorAnimacion: controlan la animación visual.

Métodos principales:

- **mover()**: desplaza el disparo hacia abajo.
- **dibujar()**: dibuja el proyectil.
- **fueraDePantalla()**: verifica si salió de los límites visibles.
- **colisionaConPrincesa()**: detecta impactos sobre la protagonista.
- **colisionaConIsla()**: detecta impactos contra plataformas.
- **moverConMapa()**: permite que el disparo acompañe el movimiento del escenario.

Problemas y soluciones:

El principal inconveniente fue lograr que los disparos permanecieran en la posición correcta cuando el mapa se desplazaba. Para resolverlo se agregó un método que modifica su posición junto con el movimiento general del escenario.

Clase Jefe

La clase **Jefe** representa al enemigo final del juego. Posee comportamientos más complejos que los enemigos comunes, incluyendo persecución, saltos entre plataformas y ataques especiales.

Variables de instancia:

- x, y: posición actual.
- ancho, alto: tamaño del jefe.
- velocidadY y gravedad: controlan la física y los saltos.
- tocandoSuelo: indica si está apoyado sobre una plataforma.
- enojado: determina si se encuentra en modo ataque.
- vaDerecha: dirección de movimiento.
- tiempoVida: tiempo máximo que permanece activo.

- disparos: arreglo que almacena sus proyectiles.
- Variables de animación y control de disparos.

Métodos principales:

- **actualizar()**: actualiza animaciones y temporizadores.
- **dibujar()**: muestra al jefe en pantalla.
- **seguirPrincesa()**: hace que persiga a la protagonista.
- **verificarSalto()**: decide cuándo saltar entre plataformas.
- **reaparecer()**: reposiciona al jefe si cae al vacío.
- **enojarse()**: activa el modo ataque.
- **generarDisparos()**: crea nuevos proyectiles.
- **actualizarDisparos()**: actualiza los disparos existentes.
- **colisiona()**: detecta impactos de la princesa.
- **colisionDisparoPrincesa()**: verifica si alguno de sus disparos golpeó a la protagonista.
- **moverConMapa()**: permite que el jefe se desplace junto con el escenario.

Problemas y soluciones:

El mayor desafío fue desarrollar una inteligencia de movimiento que permitiera al jefe desplazarse por las plataformas sin quedar atascado ni caer constantemente al vacío. Para resolverlo se implementó un sistema de detección de suelo, gravedad, saltos automáticos y reaparición en una plataforma cercana cuando abandona los límites del mapa.

Clase ReproductorDeAudio

Esta clase se encarga únicamente de reproducir la música o efectos de sonido que queramos aplicar a distintos momentos en el juego. Se implementó fuera de la clase Juego para mantener la prolijidad y no recargar la clase principal.

Variables de instancia:

- clip: almacena el audio en formato .wav a reproducir o detener.

Métodos principales:

- **reproducirMusica(ruta)**: reproduce la canción que se pase por parámetro.

- **detenerMusica()** : detiene la canción para que no se superpongan los audios.

Problemas y soluciones:

El mayor problema fue la superposición de los audios en el medio del juego o ponerlo en el tick, que hacía que se reproduzca muchas veces por segundo y colapse el juego. Se solucionó implementando los indicadores booleanos de la música por estado.

Implementación

Enemigos.java

Java

```
public static Enemigos generarEnemigo(Isla[] islas, Princesa princesa,
Enemigos[] enemigos) {
    Random r = new Random();          // Genera un enemigo sobre una isla
    aleatoria
    Isla isla = null;
    while (isla == null) {
        Isla candidata = islas[r.nextInt(islas.length)];
        if (candidata != null && !princesa.paradaSobreIsla(candidata)) {
            isla = candidata;
        }
    }
    double x = isla.getArea().x + r.nextInt(isla.getArea().width);
    // Posición aleatoria sobre la isla elegida
    double y = isla.getArea().y - 30;

    for (int i = 0; i < enemigos.length; i++) {
        // Evita que aparezcan enemigos muy juntos
        if (enemigos[i] != null) {
            if (Math.abs(x - enemigos[i].getX()) < 200) {
                return generarEnemigo(islas, princesa, enemigos);
                // Intenta generar otro enemigo
            }
        }
    }
}
```

```

        return new Enemigos(x, y, r.nextBoolean());
    }

    public boolean vaAChocar(Isla isla) {
        Rectangle futuro;
        if (vaDerecha) {
            futuro = new Rectangle((int)(x + velocidad - ancho/2), (int)(y
- alto/2), ancho, alto);
        } else {
            futuro = new Rectangle((int)(x - velocidad - ancho/2), (int)(y
- alto/2), ancho, alto);
        }
        return futuro.intersects(isla.getArea());
    }

```

generarEnemigo(): Este método se encarga de crear nuevos enemigos en el mapa. Para hacerlo, primero elige una isla al azar y verifica que la princesa no esté sobre ella, después genera una posición aleatoria dentro de esa isla para que el enemigo aparezca allí.

Además, se implementó que no pueda generarse un enemigo en un rango de 200 pixeles si hay otro cerca ya que esto fue un problema porque todos los enemigos se generaban juntos y quedaba estéticamente mal.

vaAChocar(): Este método se agregó porque los enemigos se chocaban con las islas y a veces quedaban trabados. Lo que hace es calcular la posición que tendrá el enemigo en el siguiente movimiento y comprobar si chocará con una isla. Si detecta una colisión próxima, permite que el enemigo cambie de dirección antes de chocar, evitando que se bugee.

Disparo.java

```

Java
// Calcula el ángulo entre el origen y el destino
this.angulo = Math.atan2(destinoY - y, destinoX - x);
// Dibuja el disparo en pantalla
public void dibujar(Entorno e) {

```

```

        // Si tiene frames, es un disparo especial
        if (frames != null) {
            e.dibujarImagen(frames[frameActual], x, y, angulo, 1);
        } else {
            // Dibuja el disparo normal
            e.dibujarImagen(imagen, x, y, angulo, 0.2);
        }
    }
}

```

Se implementó este código ángulo para que el disparo pueda apuntar al objetivo, y en el método *dibujar()* se muestre el proyectil en pantalla, permitiendo que la imagen o animación se pueda rotar según la dirección hacia donde se apuntó

Jefe.java

Java

```

// Hace que el jefe persiga a la princesa
public void seguirPrincesa(Princesa princesa, Isla[] islas) {
    double velocidad = 1;
    if (princesa.getX() > x) {
        x += velocidad;
        vaDerecha = true;
    }
    else if (princesa.getX() < x) {
        x -= velocidad;
        vaDerecha = false;
    }
    tocandoSuelo = false;
    // Verifica si está parado sobre una isla
    for (int i = 0; i < islas.length; i++) {
        if (islas[i] != null) {
            double techoIsla = islas[i].getY() - islas[i].getAlto()/2 - alto/2;
            boolean encimaDeIsla = x >= islas[i].getX() - islas[i].getAncho()/2
            && x <= islas[i].getX() + islas[i].getAncho()/2;

            if (encimaDeIsla && y + alto/2 >= islas[i].getY() -
            islas[i].getAlto()/2 && y < islas[i].getY()) {
                // Se apoya sobre la isla
                y = techoIsla;
                velocidadY = 0;
                tocandoSuelo = true;
            }
        }
    }
}

```

```

        break;
    }
}
}
if (!tocandoSuelo) { // Si no tiene piso debajo, cae
    velocidadY += gravedad;
    y += velocidadY;
}
if (y > 500) { // Si cae al vacío reaparece
    reaparecer(islas, princesa);
}
}

public void verificarSalto(Isla[] islas) { // Hace saltar al jefe cuando se
termina una isla
    if (!tocandoSuelo) {
        return;
    }
    boolean hayPisoAdelante = false;
    double adelante;

    if (vaDerecha) {
        adelante = x + 40;
    } else {
        adelante = x - 40;
    }

    for (int i = 0; i < islas.length; i++) { // Busca si existe una
isla adelante
        if (islas[i] != null) {
            if (adelante >= islas[i].getX() - islas[i].getAncho()/2 &&
                adelante <= islas[i].getX() + islas[i].getAncho()/2) {
                hayPisoAdelante = true;
                break;
            }
        }
    }
    if (!hayPisoAdelante) { // Si no hay piso, salta
        velocidadY = -9;
    }
}
}

```

seguirPrincesa(): este método hace que el jefe siga a la princesa verificando constantemente la interacción correctamente con las plataformas y la gravedad. También se le agregó que si este se cae al vacío aparezca en una isla cercana.

verificarSalto(): este método permite que el jefe detecte el final de una plataforma y salte automáticamente para seguir persiguiendo a la princesa sin caer al vacío. Esto fue un problema ya que el jefe al no poder saltar correctamente no se perdía en el vacío y no se podía generar otro.

Poder.java

```
Java
this.tipo = (int)(Math.random() * 2);           // 50% de probabilidad para cada uno

public void dibujar(Entorno entorno) {
    contadorAnimacion++;
    if (contadorAnimacion >= 10) {               // Cada 10 ciclos cambia la imagen.
        frameActual = (frameActual + 1) % 4; // Avanza al siguiente frame. El %
        4 hace que vuelva a empezar al llegar al último.
        contadorAnimacion = 0;
    }
    // Dibuja la imagen actual.
    double escala;
    if (tipo == 0) {
        escala = 1;
    } else {
        escala = 2.5;
    }
    entorno.dibujarImagen(frames[frameActual], x, y, 0, escala);
}
```

El constructor asigna aleatoriamente uno de los dos tipos de poder disponibles. El método *dibujar()* actualiza la animación cambiando de imagen cada ciertos ciclos y muestra el poder en pantalla con un tamaño diferente según el tipo que haya sido generado.

Conclusiones

Para concluir este informe podemos decir que este trabajo nos deja distintas formas de cómo se podría afrontar la creación de un juego en java, y distintas formas de cómo trabajar y manipular objetos en dicho lenguaje. Gracias a este tp, lograremos mejorar el uso de las herramientas que nos brinda este lenguaje, y podremos aplicar nuevas formas de encarar un problema que se nos pueda presentar.